Software Architecture and Design Specification

Railway Reservation System (Booking & Cancellation)

- **Project:** Railway Reservation System (Booking & Cancellation)
- **Version:** 1.0
- **Authors:** Team 15 (Pranav B N, Samaresh, Amruta Karoshi, Manapriya Kudurebyalya)
- **Date:** 08-10-2026
- **Status:** Draft
- **Reference documents:** SRS v1.0 (13-09-2026), STP v1.0 (16-09-2026)

Revision History

Version	Date	Author	Change Summary
0.1	08-10-2026	Team 15	Initial architecture draft from SAD template
1.0	08-10-2026	Team 15	Sprint 1 architecture and design aligned with implemented code

Approvals

Role	Name	Signature/Date
Dev Lead	Amruta Karoshi	
QA Lead	Manapriya Kudurebyalya	
Test Lead	Pranav B N	
Product Owner	Samaresh	
Instructor	Prof. M. Sheela Devi	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Railway Reservation System (RRS), a C++17 console application for booking and cancelling train tickets. It explains how the system is structured, why the main design choices were made, and how the design maps to the requirements in the SRS.

1.2 Scope

Covers passenger services: registration, login, train search with seat availability, ticket booking with PNR generation and RAC/waiting-list handling, simulated payment,

cancellation with refund calculation and waiting-list promotion, PNR status lookup, and file-based data persistence. The admin console (train, fare and quota management, reporting, backup/restore, admin action log) is designed at interface level here and will be implemented in Sprint 2.

Out of scope (as in the SRS): real payment gateway, multi-user concurrency, networked operation, mobile apps, integration with real IRCTC systems.

1.3 Audience

Developers, QA engineers, the course instructor and evaluators, and anyone maintaining or extending the system.

1.4 Definitions

PNR (Passenger Name Record), RAC (Reservation Against Cancellation), WL (Waitlist), CLI (Command Line Interface), SRS, STP, RTM, FR/NFR/SR (functional, non-functional, security requirement), ADR (Architecture Decision Record), SHA-256 (cryptographic hash), Luhn (card-number checksum).

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, security). Section 4 describes the detailed design (sequence diagrams, interface definitions, error handling, UX). Section 5 contains the glossary, references and tools. Architecture decisions are recorded as ADRs in Section 3.5.

2.2 Related Documents

Software Requirements Specification (SRS v1.0), Software Test Plan (STP v1.0), Requirements Traceability Matrix (SRS Section 8), docs/SPRINT1_PLAN.md and docs/TEST_TRACEABILITY.md in the project repository.

3. Architecture

3.1 Goals & Constraints

Goals: correctness of bookings (no double-allocation of seats, unique PNRs), data safety across crashes and restarts, security of credentials, testability of every business rule, and a simple, modular codebase (RRS-NF-005).

Constraints: C/C++17 console application; single user, single process; no external database or network (SRS); runs on Windows and Linux with GCC/G++; local files only; search response time of 1 second or less for up to 5,000 trains (RRS-NF-001); no crash on invalid input (RRS-NF-002).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Passengers	Easy menus, clear error messages, correct fares and refunds, secure login
Admin / railway clerk	Maintainable train, fare and quota data; reports (Sprint 2)

Stakeholder	Concerns
Course evaluators / instructor	Traceability to SRS, test evidence, working CI, individual contributions
Developers	Modularity, low coupling, easy unit testing
QA team	Deterministic behaviour, injectable time source, seeded test data

3.3 Component (UML) Diagram

Figure 1 shows the main components and the allowed dependency directions. The Console UI calls services; services use the domain layer; only the Storage component touches the files.

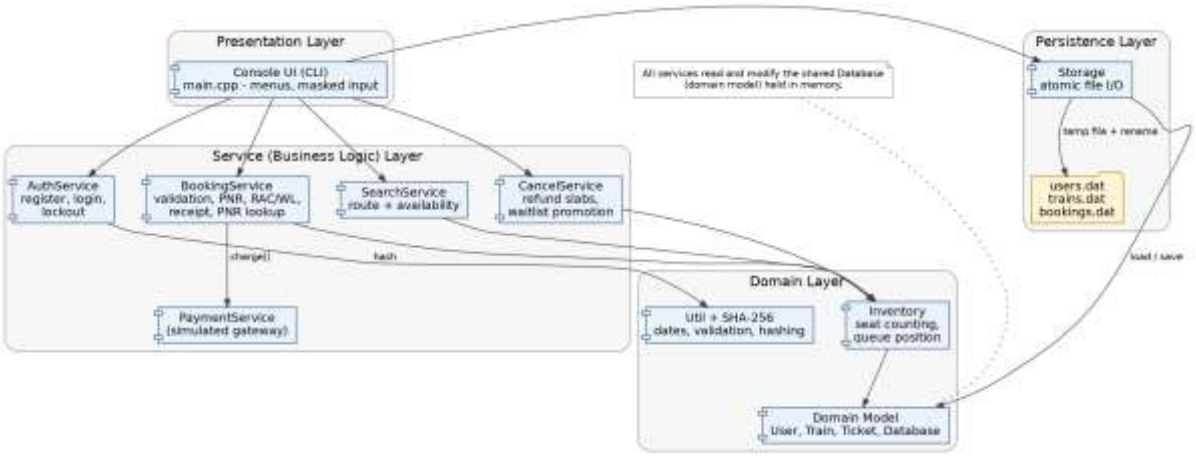


Figure 1: Component diagram of the Railway Reservation System

3.4 Component Descriptions

Component	Responsibility
Console UI (main.cpp)	Menus, prompts, masked entry of passwords and card numbers, display of tables and e-tickets. Contains no business rules.
AuthService	Registration rules, login, admin check, SHA-256 + salt hashing, lockout after 5 failed attempts for 15 minutes.
SearchService	Validates the query and journey date; lists trains and classes for a route with AVAILABLE / RAC / WL / REGRET hint.
BookingService	Validates user, train, class, date and passengers; decides CONFIRMED / RAC / WL; calls payment; creates the ticket and unique PNR; produces the e-ticket receipt; PNR status lookup.
PaymentService	Simulated gateway: 16-digit and Luhn validation, deterministic decline card, transaction id. No real bank integration.
CancelService	Cancellation by PNR with ownership check, refund slab calculation, FIFO promotion RAC to CONFIRMED and WL to RAC/CONFIRMED.
Inventory	Derives seat usage and queue positions from the ticket list, so counters can never drift out of sync with stored bookings.
Domain model (types)	User, Train, ClassInfo, Passenger, Ticket, Database and lookup helpers.
Util and SHA-256	Strict date/time parsing, journey-date window, string helpers, SHA-256 implementation.

Component	Responsibility
Storage	Loads and saves users.dat, trains.dat, bookings.dat; atomic writes (temp file then rename); skips corrupt records instead of crashing.
AdminService (Sprint 2)	Train, fare and quota management, reports, backup/restore, admin action log (RRS-F-040..043, F-051, SR-004).

3.5 Chosen Architecture Pattern and Rationale

A layered architecture (Presentation, Service, Domain, Persistence) was chosen. It separates concerns, lets each service be unit-tested without the UI or the disk, and keeps dependencies pointing downwards only. Client-server and microservice styles were rejected because the SRS defines a single-user, single-process console program; they would add network and deployment complexity with no benefit.

Architecture Decision Records

ADR	Decision	Rationale	Alternative rejected
ADR-1	Layered, modular C++17 library plus thin CLI	Testability and the SRS modularity requirement (RRS-NF-005)	Single large main.cpp
ADR-2	Seat counts derived from tickets (not stored counters)	No inconsistency after reload, crash or cancellation	Stored per-train seat counters
ADR-3	Pipe-delimited text files with atomic save (temp file + rename)	No external DB allowed; a crash cannot leave a half-written file (RRS-NF-003)	Binary structs; in-place overwrite
ADR-4	Injected clock (std::function) in all time-dependent services	Makes refund slabs, lockout and departure checks deterministic in tests	Calling the system time directly
ADR-5	SHA-256 with per-user random salt for passwords	Satisfies RRS-SR-001 without third-party libraries	Plaintext; unsalted hash
ADR-6	PNR derived from an ever-increasing booking sequence; cancelled tickets retained	PNRs never collide or get reused, even after reload (STP risk)	Random PNR with collision checks
ADR-7	One status per PNR (all-or-nothing) and strict FIFO promotion	Simple, predictable and fair behaviour that is easy to test	Splitting a booking across statuses

3.6 Technology Stack & Data Stores

- Language and standard: C++17 (GCC/G++, also builds with MSVC).
- Build: CMake (and a plain Makefile). Tests: dependency-free in-house framework, run through CTest.
- CI/CD: GitHub Actions - build and test on Ubuntu and Windows, AddressSanitizer/UBSan job, Valgrind job.
- Data stores: local text files - users.dat (accounts and lockout counters), trains.dat (trains, classes, fares, quotas), bookings.dat (tickets including cancelled ones); admin_log.txt in Sprint 2.
- Debug and quality tools: GDB, Valgrind, sanitizers, Git/GitHub Issues.

3.7 Risks & Mitigations

Risk	Mitigation
File corruption on abrupt termination (RRS-NF-003)	Atomic save via temp file and rename; corrupt lines skipped on load; tests TC_Store_03/04 and IT_10/IT_12
PNR collision after reload	PNR derived from max(seq)+1 of all stored tickets including cancelled; tests IT_05/IT_06
Waiting-list promotion bugs (unfair ordering)	Single promote() routine with FIFO rule; dedicated tests TC_Cancel_06..10
Refund-policy misunderstanding	Slabs isolated in calculateRefund() with boundary tests; assumption A-3 confirmed with the product owner
Lockout bypassed by restarting the program	Failure counters and lock time are persisted; test IT_03
Search too slow for 5,000 trains	Linear scan is acceptable (well under 1 s); verified in the Sprint 2 performance test (TC-Perf-01)

3.8 Traceability to Requirements

Requirement	Description	Component / tests
RRS-F-001..003	Registration, login, admin login	AuthService; TC_Reg_*, TC_Auth_*
RRS-F-010..012	Search, availability, RAC/WL hint	SearchService, Inventory; TC_Search_*
RRS-F-020..025	Booking, PNR, seats, RAC/WL, payment, receipt	BookingService, PaymentService; TC_Book_*
RRS-F-030..034	Cancel, refund, promotion, retention, PNR status	CancelService, BookingService; TC_Cancel_*, IT_07
RRS-F-040..043	Admin train/fare/quota, reports	AdminService (Sprint 2)
RRS-F-050/051	Persistence, backup/restore	Storage (050 in Sprint 1; 051 in Sprint 2)
RRS-NF-002/003	No crash on bad input; consistency after termination	Validation in every service; Storage; IT_10, IT_12
RRS-SR-001..005	Hashing, masking, admin access, admin log, lockout	AuthService, Console UI; TC_Sec_*; SR-004 in Sprint 2

3.9 Security Architecture

Threat modelling (STRIDE) adapted to a local console application:

Threat	Scenario	Mitigation
Spoofing	Guessing another user's password	Salted SHA-256 hashes; generic error message (no user enumeration); lockout after 5 failures
Tampering	Editing bookings.dat or users.dat by hand	Records validated on load, corrupt lines skipped; atomic writes. (Integrity signing noted as future work)

Threat	Scenario	Mitigation
Repudiation	Admin denies changing a fare	Admin action log with timestamps (RRS-SR-004, Sprint 2)
Information disclosure	Reading passwords or card numbers from files or screen	Only hashes stored; card number never stored; password and card input masked; card shown masked
Denial of service	Repeated bad input or logins	Strict input validation; lockout cool-down; corrupt data cannot crash startup
Elevation of privilege	Passenger reaching admin functions or cancelling someone else's ticket	Admin flag checked at login; ownership check in cancel(); admin menu unreachable without admin login

4. Design

4.1 Design Overview

The system is a library of small services over one in-memory Database object, plus a thin console front-end. The program loads the files at start-up, performs operations through services, and saves after every state-changing action. Every business rule lives in a service; the UI only collects input and prints results. Services return result structures (ok, message, data) instead of throwing exceptions, which keeps error handling uniform and testable.

4.2 UML Sequence Diagrams

Three flows are shown: booking a ticket (Figure 2), cancelling a ticket with refund and waiting-list promotion (Figure 3) and login with lockout (Figure 4).

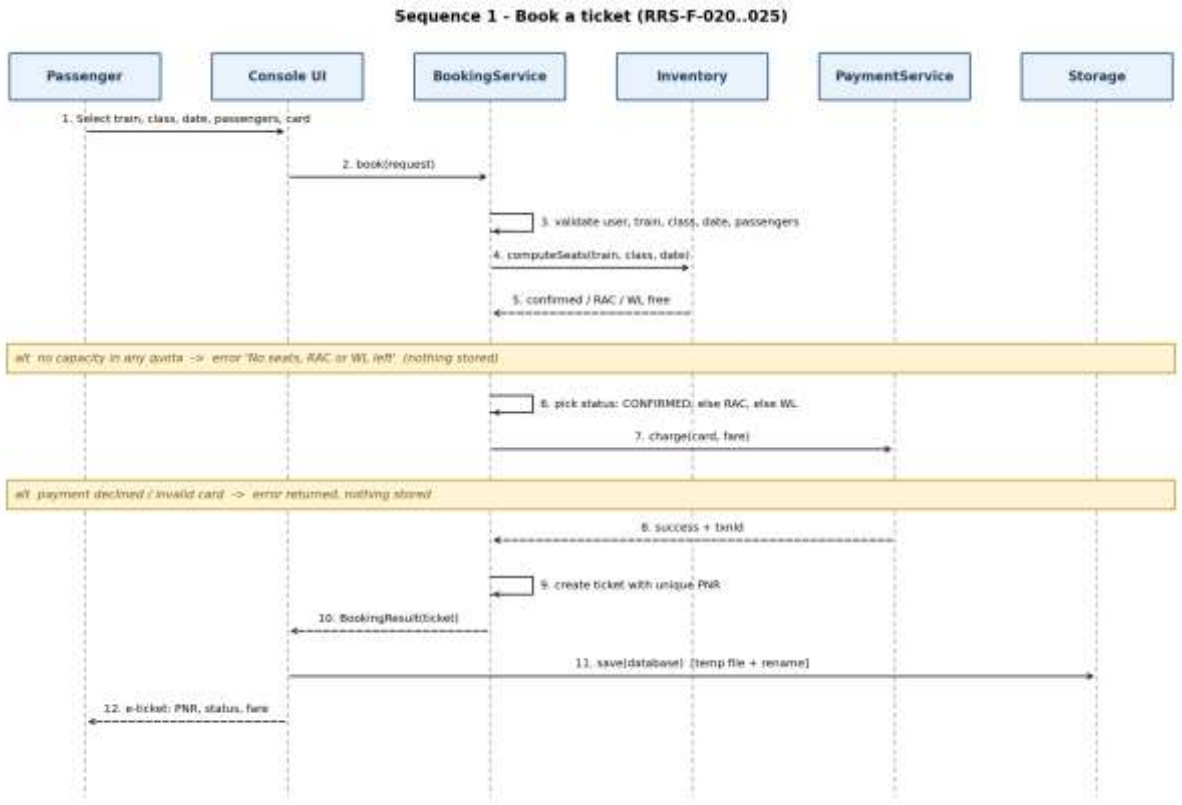


Figure 2: Sequence diagram - book a ticket

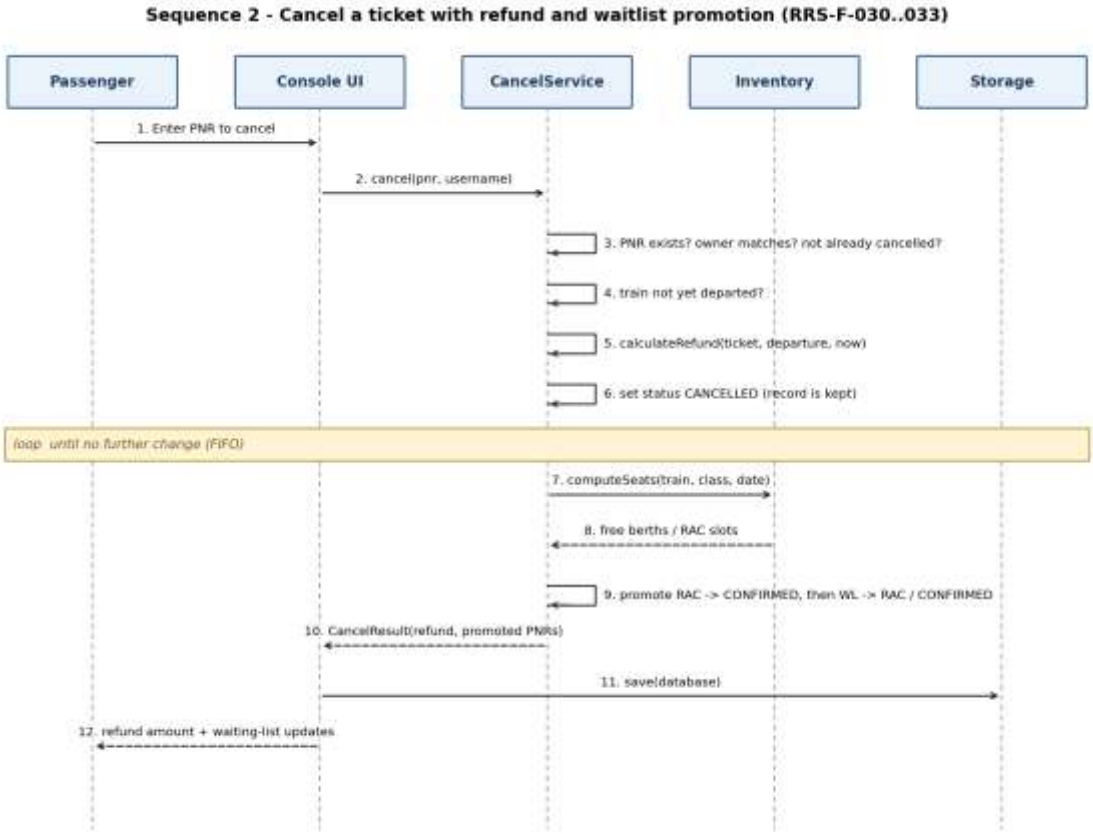


Figure 3: Sequence diagram - cancel a ticket and promote the waiting list

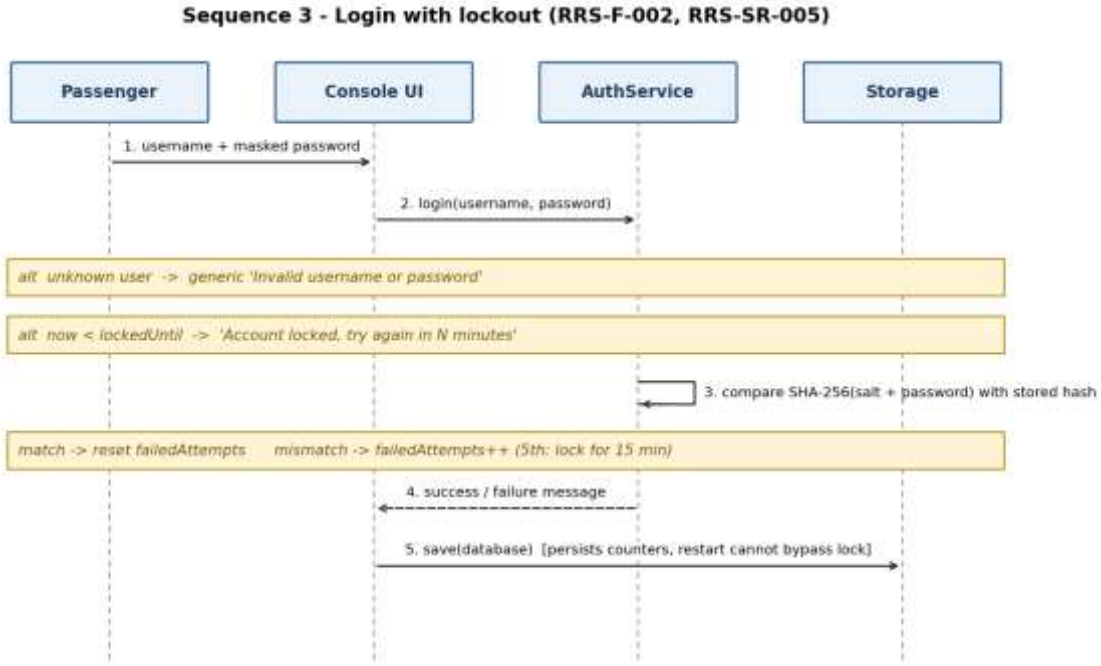


Figure 4: Sequence diagram - login with lockout

4.3 API Design

The components expose C++ interfaces rather than network endpoints. The three main ones are defined below.

Interface 1 - BookingService::book(BookingRequest) -> BookingResult

- Request: { username, trainNumber, classCode, journeyDate (YYYY-MM-DD), passengers[1..6] {name, age 1..120, gender M/F/O}, cardNumber }
- Response: { ok, message, ticket {pnr, status CONFIRMED/RAC/WL, totalFare, ...}, txnId }
- Errors (ok = false, nothing stored): unknown user; train not found or inactive; class not on train; invalid or past journey date; train already departed; invalid passenger count or details; no seat/RAC/WL capacity; payment declined or invalid card.

Interface 2 - CancelService::cancel(pnr, requester) -> CancelResult

- Request: { pnr, requester username }
- Response: { ok, message, refund (rupees), promotedPnrs[] }
- Errors: PNR not found; requester is not the owner; already cancelled; train already departed.
- Refund slabs: WL 100%; 48 h or more: 100% minus Rs.60 per passenger; 12-48 h: 75%; 4-12 h: 50%; under 4 h: 0%.

Interface 3 - AuthService::login(username, password, adminLogin) -> AuthResult

- Response: { ok, message }. Errors: invalid username or password (same text for both cases), account locked (with minutes remaining), administrator account required.

Other interfaces: SearchService::search(SearchQuery) -> SearchResult {rows[]}; lookupPnr(db, pnr) -> Ticket*; PaymentService::charge(card, amount) -> PaymentResult; Storage::load(Database&, LoadReport*) and Storage::save(const Database&, error*).

4.4 Error Handling, Logging & Monitoring

- All user input is validated before use; invalid input produces a plain-language message and never a crash (RRS-NF-002).
- Services return result structures with a user-safe message; no stack traces or internal paths are shown to the user.
- Messages never reveal sensitive data: passwords and full card numbers are never printed or stored.
- Corrupt data records are counted and skipped at load; the CLI reports how many were skipped.
- Admin actions will be appended with a timestamp to admin_log.txt (RRS-SR-004, Sprint 2).
- Monitoring for this project is done through the test suite and CI results (pass rate per module), as defined in the STP metrics.

4.5 UX Design

Plain-language numbered menus (RRS-NF-004). Passwords and card numbers are not echoed while typing. Search results are shown as an aligned table with an availability hint (AVAILABLE n / RAC n / WL n / REGRET). The e-ticket shows PNR, train, route, date, class, status with queue position for RAC/WL, passengers and fare. Invalid menu choices show a short instruction and re-display the menu. Accessibility is limited to a console UI (no GUI in this version).

4.6 Open Issues & Next Steps

- Sprint 2: admin console (train, fare, quota), reports, backup/restore, admin action log, performance test with 5,000 trains, fuzz and system tests.
- Confirm the refund slabs and quota sizes against the final SRS wording.
- Future: integrity protection for data files, multi-user access with file locking or a database, real payment gateway, GUI/web front-end.

5. Appendices

5.1 Glossary

PNR, RAC, WL, CLI, SRS, STP, RTM, FR/NFR/SR, ADR, SHA-256, Luhn - see Section 1.4.

5.2 References

IEEE 42010 (architecture description), OWASP guidance on password storage and input validation, NIST SP 800-160, FIPS 180-4 (SHA-256), SRS v1.0, STP v1.0.

5.3 Tools

Graphviz (component diagram), matplotlib/PlantUML/draw.io (sequence diagrams), CMake, GitHub Actions, Valgrind, GDB.